

Assignment Kit for Coding Standard



Personal Software Process for Engineers: Part I

The Software Engineering Institute (SEI)
is a federally funded research and development center
sponsored by the U.S. Department of Defense and
operated by Carnegie Mellon University.

This material is approved for public release.
Distribution limited by the Software Engineering Institute to attendees.

Personal Software Process for Engineers: Part I

Assignment Kit for the Coding Standard

Overview

Overview This assignment kit covers the following topics.

Section	See Page
Prerequisites	2
Objectives	2
Coding standard requirements	3
Example coding standard	4
Evaluation criteria and suggestions	7
Coding standard template	8

Prerequisites

Prerequisites

- Read Chapter 4
- Complete Size Counting Standard

Objectives

The objectives of the coding standard are to

- establish a consistent set of coding practices
- provide criteria for judging the quality of the code that you produce
- facilitate size counting by ensuring your programs are written so they can be readily counted
- for LOC counting, require that there be a separate physical line for each logical line of code

Coding standard requirements

Coding standard requirements

Produce, document, and submit a completed coding standard that calls for quality coding practices.

For LOC counting, ensure that a separate physical source line is used for each logical line of code.

Submit the coding standard with your program 2 assignment package.

Example coding Standard

Coding standard example

Pages 5 and 6 of this workbook contain an example C++ coding standard.

Notes about the example

- Since it is an example, tailor it to meet your personal needs.
 - If you have an existing organizational standard, consider using it for the PSP exercises.
-

Continued on next page

Example C++ Coding Standard

Purpose	To guide implementation of C++ programs
Program Headers	Begin all programs with a descriptive header.
Header Format	<pre> /***** /* Program Assignment: the program number */ /* Name: your name */ /* Date: the date you started developing the program */ /* Description: a short description of the program and what it does */ *****/ </pre>
Listing Contents	Provide a summary of the listing contents
Contents Example	<pre> /***** /* Listing Contents: */ /* Reuse instructions */ /* Modification instructions */ /* Compilation instructions */ /* Includes */ /* Class declarations: */ /* CData */ /* ASet */ /* Source code in c:/classes/CData.cpp: */ /* CData */ /* CData() */ /* Empty() */ *****/ </pre>

(continued)

Example C++ Coding Standard (continued)

Reuse Instructions	- Describe how the program is used: declaration format, parameter values, types, and formats. - Provide warnings of illegal values, overflow conditions, or other conditions that could potentially result in improper operation.
Reuse Instruction Example	<pre> /***** /* Reuse instructions */ /* int PrintLine(char *line_of_character) */ /* Purpose: to print string, 'line_of_character', on one print line */ /* Limitations: the line length must not exceed LINE_LENGTH */ /* Return 0 if printer not ready to print, else 1 */ *****/ </pre>
Identifiers	Use descriptive names for all variable, function names, constants, and other identifiers. Avoid abbreviations or single-letter variables.
Identifier Example	<pre> Int number_of_students; /* This is GOOD */ Float: x4, j, ftave; /* This is BAD */ </pre>
Comments	- Document the code so the reader can understand its operation. - Comments should explain both the purpose and behavior of the code. - Comment variable declarations to indicate their purpose.
Good Comment	<pre>If(record_count > limit) /* have all records been processed? */</pre>
Bad Comment	<pre>If(record_count > limit) /* check if record count exceeds limit */</pre>
Major Sections	Precede major program sections by a block comment that describes the processing done in the next section.
Example	<pre> /***** /* The program section examines the contents of the array 'grades' and calcu- */ /* lates the average class grade. */ *****/ </pre>
Blank Spaces	- Write programs with sufficient spacing so they do not appear crowded. - Separate every program construct with at least one space.
Indenting	- Indent each brace level from the preceding level. - Open and close braces should be on lines by themselves and aligned.
Indenting Example	<pre> while (miss_distance > threshold) { success_code = move_robot (target_location); if (success_code == MOVE_FAILED) { printf("The robot move has failed.\n"); } } </pre>
Capitalization	- Capitalize all defines. - Lowercase all other identifiers and reserved words. - To make them readable, user messages may use mixed case.
Capitalization Examples	<pre> #define DEFAULT-NUMBER-OF-STUDENTS 15 int class-size = DEFAULT-NUMBER-OF-STUDENTS; </pre>

Evaluation criteria and suggestions

Evaluation criteria

Your standard must be

- complete
 - legible
-

Suggestions

Keep your standards simple and short.

Do not hesitate to copy or build on the PSP materials.

Coding Standard Template

Purpose	Guiar el desarrollo de programas Java para cálculo de integral t-Student con Regla de Simpson.
Program Headers	Comenzar todos los programas con un encabezado descriptivo.
Header Format	<pre>java /***** Programa: Regla de Simpson para Distribución t-Student * Clase: [NombreClase] * Autor: Karla Sofía Castro Pérez * Fecha: 4-Diciembre-2024 * * Descripción: [Descripción breve de la funcionalidad] *****/</pre>
Listing Contents	Proporcionar un resumen del contenido del listado después del encabezado.
Contents Example	<pre>/* Reutilización: Ejecutar: java App */ (como en SimpsonIntegration.java)</pre>
Reuse Instructions	• Describir cómo se usa el programa: entrada por consola, parámetros. • Advertir sobre valores no válidos: dof > 0, segmentos pares.
Reuse Example	<pre>java /* Uso: Para calcular la integral $P = \int [x_{Inicial}, x_{Final}] f(t) dt$: double p = SimpsonIntegration.calculateP(xInicial, xFinal, numSeg, dof); Advertencias: - numSeg debe ser número par - La precisión mejora al aumentar numSeg */</pre>
Identifiers	Usar nombres descriptivos con prefijos de tipo: dbl para double, int para int, obj para objetos.
Identifier Example	<pre>double dblXInicial; (BIEN) int intDof; (BIEN) double x; (MAL - no sigue convención)</pre>

(continued)

Coding Standard Template (continued)

Comments	• Documentar cada método con comentarios Javadoc (<code>/** */</code>). Usar comentarios de bloque (<code>/* */</code>) para secciones principales.
Good Comment	<code>java /** * Calcula el valor de la función de densidad de probabilidad * t-Student en un punto dado. *</code>
Bad Comment	<code>// calcula t</code>
Major Sections	Preceder secciones principales del programa con un comentario de bloque con separadores.
Example	<code>java /*-----*/ /* Validación de parámetros */ /*-----*/</code>
Blank Spaces	Separar secciones con líneas en blanco. Dejar un espacio después de if, for, while.
Indenting	• Usar 4 espacios por nivel de indentación. Abrir llaves en la misma línea, cerrar en línea separada alineada.
Indenting Example	<code>java if (intDof <= 0) { throw new IllegalArgumentException("tDistribution: dof debe ser > 0. Valor recibido: " + intDof); }</code>
Capitalization	• Nombres de clase en PascalCase: SimpsonIntegration • Variables en camelCase con prefijo: dblXFinal • Constantes en MAYÚSCULAS: no se usan en este código.
Capitalization Example	<code>public class GammaFunction private double dblIPFinal;</code>